

Homework — 2.1.5 Thinking Concurrently — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Mark scheme

Part A (14 marks)

1. (a) [2] 1 mark: each frame is computed from its own data and does **not** depend on the result of any other frame — there is no data dependency between frames. 1 mark: the frames can therefore be shared out and processed **at the same time** without one having to wait for another, then collected together at the end.

(b) [1] Any one: a multi-core processor / multiple processors / a GPU / a render farm of several networked machines.

2. [2] 1 mark: the spell-check runs as a separate task in the background at the same time as editing. 1 mark: so the program stays **responsive** — the user can keep typing instead of waiting for the long spell-check to finish.

3. (a) [1] A **race condition**.

(b) [2] 1 mark: both processes accessed the **same shared data** (the stock count) and each read the value 1 **before** either had written its update. 1 mark: because the two updates were not coordinated, the final result depended on the unpredictable **timing/order** of the operations, allowing the last item to be sold twice.

4. [2] Any two (1 each): on a **single-core** processor the tasks are only time-sliced, not truly parallel, so there may be no real speed-up; there is an **overhead** in creating, coordinating and recombining concurrent tasks, which can outweigh the gain; some problems are **inherently sequential** — later steps depend on earlier outputs, so they cannot run at the same time.

5. [4] Up to 4 marks: - (b) reserving the flight and (c) reserving the hotel are **independent** of each other, so they could run **concurrently (1)**. - A correct dependency, e.g. (d) the confirmation email must be **sequenced after** the payment/reservations because it reports their outcome (accept: the card should be charged before/alongside reservations, with the email last) **(1)**. - Benefit **(1)**: performing the independent reservations at the same time makes the booking complete faster / keeps the site responsive for the customer. - Trade-off **(1)**: concurrent tasks are harder to design/test/debug; if tasks share data (e.g. the same last seat) a **race condition** could occur; results must be brought back together before confirming; on limited hardware there may be no real speed-up. - Must be applied to the booking scenario and include both a benefit and a trade-off for full marks.

Part B (6 marks)

6. [2] 1 mark: both conditions correct — `movesLeft = 0` (or `movesLeft == 0` / `moves <= 0`) and `score >= 100`. 1 mark: correctly combined with the **OR** operator, e.g. `movesLeft = 0 OR score >= 100`. Must be a valid Boolean expression.

7. [2] All four rows correct = 2 marks; any three rows correct = 1 mark:

A	B	Q
0	0	0
0	1	0
1	0	1
1	1	0

8. [2] 1 mark + 1 mark for a clear paired difference: a **static** structure has a **fixed size** set when it is created (e.g. an array) [1]; a **dynamic** structure can **grow and shrink** at run time as items are added or removed (e.g. a linked list) [1].

Total: 14 + 6 = 20 marks